

Recognizing 20 Cat and Dog Breeds with Detector-Guided Crops, Domain-Randomized Training, and a Confidence-Pattern Meta-Classifer

Uğur Cem Sağlam Umut Kara Doruk Berberoğlu Yesienii Liaskina
Bachelor Informatik (PO 2022), Ludwig-Maximilians-Universität München

{U.Saglam, Umut.Kara, D.Berberoglu, Yesienii.Liaskina}@campus.lmu.de

Abstract

We tackle fine-grained recognition of pet photos: given an image, our system returns one of 20 cat and dog breeds, or a reject label (-1) when the image shows no target animal. Fine-grained recognition is hard because two breeds can look almost identical while one breed varies widely in pose, lighting, and background, and small datasets tempt a network to memorize pixels instead of learning the breed. We build a three-part pipeline. An off-the-shelf object detector first localizes and crops the largest animal, so the classifier sees a clean, single-animal image and we get a natural way to reject pictures with no animal. The classifier is then trained under heavy, randomized image corruption (blur, pixelation, color and aspect changes, noise) so that it leans on stable breed structure (shape and proportions) rather than memorized texture. Finally, a lightweight meta-classifier reads the base model’s confidence across many fixed views of the same crop and learns to both sharpen the prediction and flag unreliable ones, which also drives the reject decision. This report reviews the relevant work and data, and lays out our candidate models, training plan, and evaluation.

1. Introduction

Our task is image classification with a twist. Given a photograph of a pet, the system must output one of 20 cat and dog breeds (an index from 0 to 19), or -1 when the image contains no target animal, for example a different species, or no animal at all. It is therefore a 21-way decision: 20 breeds plus one reject class. Evaluation runs on a private test set drawn from the same distribution as a provided validation set; when several animals appear, the largest target animal by image area defines the label, and the system is expected to follow the same rule.

This is a *fine-grained* problem. Two breeds (say, two terriers) may differ only in subtle coat or muzzle cues, while a single breed shows up in very different poses, crops, and

lighting. With only a modest number of images per class, a large network can simply memorize the training set: it reaches low training loss but learns the wrong cues and generalizes poorly.

Our main idea is to stop the network from relying on memorized texture. During training we randomly corrupt the appearance of every image (blur, pixelation, color shifts, aspect distortion, noise) so that the cues which survive are breed structure: shape and proportion. This mirrors *domain randomization* in robotics [12], where randomizing a simulator’s textures and lighting forces a control policy to work in the real world. Here we randomize the photo so the classifier latches onto the breed concept rather than the exact pixels.

Around this classifier we add two practical pieces. An off-the-shelf detector localizes the animal and crops to it, which handles cluttered images with distractor objects and gives a clean way to reject images with no animal. A small meta-classifier then looks at how the base model behaves across many fixed views of the same crop, and learns to refine the label and to score how trustworthy the prediction is.

Our contributions for this project are: (1) a detector-guided, domain-randomized training pipeline for fine-grained pet classification; and (2) a confidence-pattern meta-classifier that aggregates predictions over many views to refine the label and to decide the reject class from cross-view disagreement.

2. Related Work

Fine-grained pet recognition. Breed recognition is the setting of the Oxford-IIIT Pet dataset [8]; our 20-breed task is comparable in scale and difficulty. The standard, data-efficient recipe is transfer learning from ImageNet-pretrained backbones such as ResNet [3] and EfficientNet [11].

Data augmentation and shape bias. Augmentation has been central to image classification since the flips and crops of AlexNet [6], and is now automated by policies like Ran-

dAugment [1]. Geirhos *et al.* [2] show that ImageNet-trained CNNs prefer texture over shape, and that texture-destroying perturbations (grayscale, blur, noise) push them toward shape, which is exactly what our regime is designed to do.

Corruption robustness. ImageNet-C [4] standardizes the corruptions we use (blur, pixelation, noise, compression). We reuse them both as training augmentations and as a held-out test of whether the model really learned the breed.

Object detection. Modern off-the-shelf detectors such as YOLO [9] localize objects well without any fine-tuning, which is all we need to crop the animal before classification.

Combining views and models. Averaging predictions over augmented views (test-time augmentation) and over models (deep ensembles [7]) reliably helps. Our meta-classifier replaces fixed averaging with a *learned* aggregation, a form of stacked generalization [13].

Knowing when to abstain. The maximum softmax probability is a simple, strong signal for spotting mistakes and out-of-distribution inputs [5]; our meta-classifier uses a richer signal (the pattern of confidences across many views) to drive the reject decision.

Explainability. To check what the model looks at, we use Grad-CAM [10] and the occlusion-style analysis of Zeiler and Fergus [14].

3. Data

The dataset covers 20 target breeds plus *confounders*, images of other animals or objects (for example, elephants) that must be rejected. The validation set is fully annotated with ground-truth bounding boxes and labels, and the private test set follows the same distribution, with at most one target animal per image and the largest-by-area rule for the label.

Before modeling we will audit the data: per-class image counts (to measure imbalance), the resolution and aspect-ratio distribution, near-duplicate images, and obvious label noise. We use the provided bounding boxes to crop animals tightly for training, so the classifier learns on the same kind of single-animal crop that the detector produces at test time. We keep a stratified split (e.g. 70/15/15) with a clean, *un-augmented* validation and test set, so that gains from our augmentation are not confused with the evaluation transform itself. All inputs are resized to the backbone resolution (224×224) with ImageNet mean/variance normalization; we will also compare letterboxing, center-cropping, and squashing.

4. Method

Figure 1 shows the full pipeline: detect and crop the animal, classify the crop under many fixed views, then aggregate the

results with a meta-classifier into a breed label or a reject.

4.1. Detector and crop

We run a pretrained, off-the-shelf detector (a YOLO model [9]) on the full image to find animals and their boxes. We keep only animal detections and discard everything else, which also saves compute by never sending background clutter to the classifier. Among the animals found, we crop to the largest by box area, matching the label rule. If no animal is found, we output -1 directly.

4.2. Stage 1: domain-randomized classifier

Our augmentation bank has three groups: *geometric* (random resized crop, horizontal flip, small rotation, aspect squash/stretch); *photometric* (brightness, contrast, saturation, and hue jitter, random grayscale); and *degradation* (Gaussian and motion blur, pixelation by down-then-up re-sampling, additive noise, JPEG artifacts). The degradation group is taken straight from ImageNet-C [4].

We treat training and inference differently. For training we do not fix a set of variants; we sample the corruption strengths randomly for each image, in each epoch, giving an effectively unlimited range of appearances. For inference we instead reserve a fixed bank of K deterministic views, so that the meta-classifier sees comparable inputs across all images.

Candidate backbones are ResNet-18 and ResNet-50 [3], with EfficientNet-B0 [11] and MobileNetV3 for an accuracy/speed comparison. As required, we also train one CNN *from random initialization* (no pretrained weights) as a baseline, which lets us measure exactly what pretraining buys us. The classifier has 21 outputs (20 breeds plus a reject slot).

4.3. Stage 2: confidence-pattern meta-classifier

At inference, a crop is passed through all K views to produce K confidence vectors. From these we build a compact feature: the per-view outputs plus simple summaries (the mean and variance across views, the per-view entropy, how often the top-1 prediction agrees, and the gap between the top two classes). A small multilayer perceptron then maps this feature to two outputs,

$$g(\phi(x)) = (\hat{y}, s), \quad (1)$$

a refined label $\hat{y}$ and a reject score $s \in [0, 1]$. The intuition is that the meta-classifier learns tell-tale patterns, for example, “confident on the clean view but collapsing under mild blur,” which signals a texture-driven, unreliable match. We keep the meta-model deliberately simple (a small MLP) and implement it in PyTorch, in line with the project’s requirements.

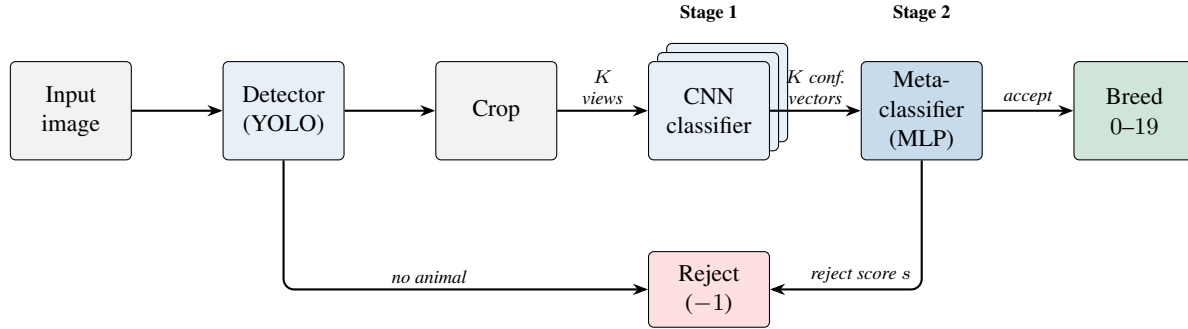


Figure 1. Overview of the pipeline. A pretrained detector localizes and crops the largest animal (or returns -1 if none is found). The crop is classified under a fixed bank of K deterministic views, and a small meta-classifier turns the resulting confidence pattern into a refined breed label or a reject decision.

4.4. Reject class (-1)

We will compare three ways to produce the reject decision: (i) a 21st “none” output on the classifier, trained on the confounder images; (ii) a separate gatekeeper model that decides animal-vs-not before the breed model runs; and (iii) the meta-classifier’s reject score, where a non-target image is simply the most extreme unreliable prediction and makes the views disagree. We expect option (iii) to work best, because it reuses machinery we already build and keeps the system as one coherent pipeline; the detector’s “no animal found” path acts as a cheap first filter regardless. Reporting all three (including the ones that do not help) is part of the analysis the project asks for.

5. Training Strategy

Stage 1. We first train the from-scratch baseline, then the transfer-learned model in two phases: (A) freeze the backbone and train only the new head, then (B) unfreeze and fine-tune the whole network at a low learning rate so it adapts to pets without forgetting useful features. We use cross-entropy with label smoothing, the AdamW optimizer, a cosine learning-rate schedule with warmup, mixed-precision training, and early stopping on validation macro-F1. Class imbalance is handled with a weighted loss or class-balanced sampling.

Stage 2. The base network is frozen while we train the meta-classifier. The key pitfall is *data leakage*: a model that has memorized its own training images gives unrealistically confident features on exactly those images. We therefore build the meta-features from out-of-fold predictions (k -fold over the training set) or from a separate held-out split, never from images the base model trained on. The reject head is supervised by whether the base top-1 prediction matched the true label.

Compute. Running K views multiplies inference time by about K , so K is our speed dial. We cap it (and keep the view set small), batch the K views in a single forward pass,

and tune K to stay under the 5-second-per-image limit.

6. Evaluation Plan and Risks

Metrics. We report exactly the metrics defined for the project: per-class precision, recall, and F1 (one-vs-rest), overall accuracy, and the macro- and weighted-average precision, recall, and F1. The reject class -1 is treated as a normal class, so wrongly accepting a confounder or wrongly rejecting a real breed is penalized. We also show the confusion matrix.

Ablation ladder. To isolate the value of each idea we evaluate: (0) no augmentation, (1) standard augmentation, (2) domain-randomized augmentation, (3) adding test-time averaging, and (4) adding the meta-classifier. A per-transform leave-one-out test will show which corruptions help and which hurt; we expect aspect-squash and extreme pixelation to be the riskiest.

Explainability. We run Grad-CAM [10] on the classifier to see which regions drive each decision. This doubles as a test of our core claim: we compare a model trained on clean crops with one trained on heavily corrupted crops, and check whether the corrupted-trained model focuses on breed structure (snout, ears) instead of background or fur texture.

Risks and mitigations. Aggressive augmentation can lower clean accuracy; we use bounded strengths and the ablation ladder to catch this. The meta-classifier could overfit or leak; we use out-of-fold features and a simple model. Compute could be tight; we cap K . A rough schedule is: data audit and baseline first, then the augmentation regime and backbone choice, then the meta-classifier, then evaluation, ablations, and write-up, adjusted to the course timeline.

References

- [1] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le. RandAugment: Practical Automated Data Augmentation with a Reduced Search Space. In *NeurIPS*, 2020. 2

- [2] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, and W. Brendel. ImageNet-trained CNNs are Biased Towards Texture; Increasing Shape Bias Improves Accuracy and Robustness. In *ICLR*, 2019. [2](#)
- [3] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *CVPR*, 2016. [1](#), [2](#)
- [4] D. Hendrycks and T. Dietterich. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. In *ICLR*, 2019. [2](#)
- [5] D. Hendrycks and K. Gimpel. A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks. In *ICLR*, 2017. [2](#)
- [6] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. In *NeurIPS*, 2012. [1](#)
- [7] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. In *NeurIPS*, 2017. [2](#)
- [8] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and Dogs. In *CVPR*, 2012. [1](#)
- [9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. In *CVPR*, 2016. [2](#)
- [10] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. In *ICCV*, 2017. [2](#), [3](#)
- [11] M. Tan and Q. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *ICML*, 2019. [1](#), [2](#)
- [12] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In *IROS*, 2017. [1](#)
- [13] D. H. Wolpert. Stacked Generalization. *Neural Networks*, 5 (2):241–259, 1992. [2](#)
- [14] M. D. Zeiler and R. Fergus. Visualizing and Understanding Convolutional Networks. In *ECCV*, 2014. [2](#)